

OS PRACTICAL 3

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

Created a C program using the `fork()` system call. The `fork()` function creates a new child process from the existing parent process.

Created the child process displayed the PID and PPID of both the parent and child processes to identify them separately.

```
rishita@Rishitha:~$ nano forkprac3.c
rishita@Rishitha:~$ gcc forkprac3.c
rishita@Rishitha:~$ ./a.out
Parent Process
Parent Process ID is 538
Parent's Parent Process ID is 469
Child Process
Child Process ID is 539
Parent Process ID is 466
rishita@Rishitha:~$ |
```

During the execution of the program, the `ps` command was used to view the processes currently running in the terminal. It displayed the process IDs and the corresponding commands

```
rishita@Rishitha:~$ ps
  PID TTY          TIME CMD
  469 pts/0        00:00:00 bash
  539 pts/0        00:04:56 a.out
  622 pts/0        00:00:02 a.out
  632 pts/0        00:00:00 ps
```

Used the `top` command to continuously monitor the processes while they were executing. It provided a real-time view of the processes and their activity. This helped in observing how the process status changes during execution.

```

632 pts/0    00:00:00 ps
rishita@Rishitha:~$ top
top - 08:21:56 up 8 min,  1 user,  load average: 1.49, 0.83, 0.37
Tasks:  27 total,   3 running,  24 sleeping,   0 stopped,   0 zombie
%Cpu(s): 16.6 us,  0.0 sy,  0.0 ni, 83.2 id,  0.0 wa,  0.0 hi,  0.2 si,  0.
MiB Mem :  7784.6 total,  6416.2 free,   532.7 used,   988.1 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  7251.9 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
539	rishita	20	0	2768	1064	968	R	100.0	0.0	5:32.80
622	rishita	20	0	2768	1068	968	R	100.0	0.0	0:38.45
1	root	20	0	24136	15332	11544	S	0.0	0.2	0:01.10
2	root	20	0	3180	2208	2072	S	0.0	0.0	0:00.01
7	root	20	0	3180	2048	1940	S	0.0	0.0	0:00.00
50	root	19	-1	50400	16836	15536	S	0.0	0.2	0:00.29
82	systemd+	20	0	22416	14440	11944	S	0.0	0.2	0:00.16
89	root	20	0	35368	12356	9264	S	0.0	0.2	0:00.65
111	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.06
112	root	20	0	4472	3124	2876	S	0.0	0.0	0:00.01
113	message+	20	0	8796	5368	4684	S	0.0	0.1	0:00.08
117	root	20	0	44096	29768	14624	S	0.0	0.4	0:00.47
131	root	20	0	18436	9412	8172	S	0.0	0.1	0:00.19
146	root	20	0	1792300	15212	12552	S	0.0	0.2	0:00.19
177	syslog	20	0	220548	5752	4504	S	0.0	0.1	0:00.11
206	_chrony	20	0	20424	6452	5676	S	0.0	0.1	0:00.05
213	_chrony	20	0	12096	2436	1792	S	0.0	0.0	0:00.00
271	root	20	0	5492	2724	2540	S	0.0	0.0	0:00.02
274	root	20	0	123460	32632	18016	S	0.0	0.4	0:00.34
321	root	20	0	8644	5492	4652	S	0.0	0.1	0:00.02
383	rishita	20	0	22348	13576	10948	S	0.0	0.2	0:00.16
385	rishita	20	0	22916	4088	2088	S	0.0	0.1	0:00.00
413	rishita	20	0	6136	5496	3928	S	0.0	0.1	0:00.03
465	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00
466	root	20	0	3200	1260	1108	S	0.0	0.0	0:00.04
469	rishita	20	0	6268	5600	3928	S	0.0	0.1	0:00.09
644	rishita	20	0	8172	6020	3912	R	0.0	0.1	0:00.00

Used the /proc filesystem to inspect the process with PID 622, which was the forkprac3 process identified using the ps command. The /proc/622 directory contains different files and directories that provide information about the process, such as its status, memory, scheduling, file descriptors and other process-related details. This helped me observe how Linux maintains information about an active process

```
rishita@Rishitha:~$ ls /proc/622
arch_status  gid_map      ns           smaps
attr         io           numa_maps   smaps_rollup
auxv         ksm_merging_pages oom_adj     stack
cgroup       ksm_stat    oom_score   stat
clear_refs   latency     oom_score_adj statm
cmdline      limits     pagemap     status
comm         loginuid   patch_state syscall
coredump_filter map_files  personality task
cpuset       maps       projid_map  timens_offsets
cwd          mem        root        timers
environ      mountinfo  sched       timerslack_ns
exe          mounts     schedstat  uid_map
fd           mountstats sessionid   wchan
fdinfo       net        setgroups
rishita@Rishitha:~$ |
```